

- ResourceConsumer/ResourceProducer – set this to ResourceConsumer
- VendorData – optional binary data whose meaning is defined by the pin muxing server. This should usually be left blank
- Pin List – a comma separated list of pin numbers to which the configuration applies. When the pin muxing server is a GpioClx driver, these are GPIO pin numbers and have the same meaning as pin numbers in a Gpiolo descriptor.

The following example shows how one might supply a MsftFunctionConfig() resource to an I2C controller driver.

C++

```
Device(I2C1)
{
    Name(_HID, "BCM2841")
    Name(_CID, "BCM2841")
    Name(_UID, 0x1)
    Method(_STA)
    {
        Return(0xf)
    }
    Method(_CRS, 0x0, NotSerialized)
    {
        Name(RBUF, ResourceTemplate()
        {
            Memory32Fixed(ReadWrite, 0x3F804000, 0x20)
            Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) { 0x55 }
            MsftFunctionConfig(Exclusive, PullUp, 4, "\\_SB.GPI0", 0,
ResourceConsumer, ) { 2, 3 }
        })
        Return(RBUF)
    }
}
```

In addition to the memory and interrupt resources typically required by a controller driver, a MsftFunctionConfig() resource is also specified. This resource enables the I2C controller driver to put pins 2 and 3 - managed by the device node at _SB.GPI00 – in function 4 with pull-up resistor enabled.

Supporting muxing support in GpioClx client drivers

GpioClx has built-in support for pin muxing. GpioClx miniport drivers (also referred to as "GpioClx client drivers"), drive GPIO controller hardware. As of Windows 10 build

14327, GpioClx miniport drivers can add support for pin muxing by implementing two new DDIs:

- CLIENT_ConnectFunctionConfigPins – called by GpioClx to command the miniport driver to apply the specified muxing configuration.
- CLIENT_DisconnectFunctionConfigPins – called by GpioClx to command the miniport driver to revert the muxing configuration.

See [GpioClx Event Callback Functions](#) for a description of these routines.

In addition to these two new DDIs, existing DDIs should be audited for pin muxing compatibility:

- CLIENT_ConnectIoPins/CLIENT_ConnectInterrupt – CLIENT_ConnectIoPins is called by GpioClx to command the miniport driver to configure a set pins for GPIO input or output. GPIO is mutually exclusive with MsftFunctionConfig, meaning a pin will never be connected for GPIO and MsftFunctionConfig at the same time. Since a pin's default function is not required to be GPIO, a pin may not necessarily not be muxed to GPIO when ConnectIoPins is called. ConnectIoPins is required to perform all operations necessary to make the pin ready for GPIO IO, including muxing operations. *CLIENT_ConnectInterrupt* should behave similarly, since interrupts can be thought of as a special case of GPIO input.
- CLIENT_DisconnectIoPins/CLIENT_DisconnectInterrupt – These routine should return pins to the state they were in when CLIENT_ConnectIoPins/CLIENT_ConnectInterrupt was called, unless the PreserveConfiguration flag is specified. In addition to reverting the direction of pins to their default state, the miniport should also revert each pin's muxing state to the state it was in when the _Connect routine was called.

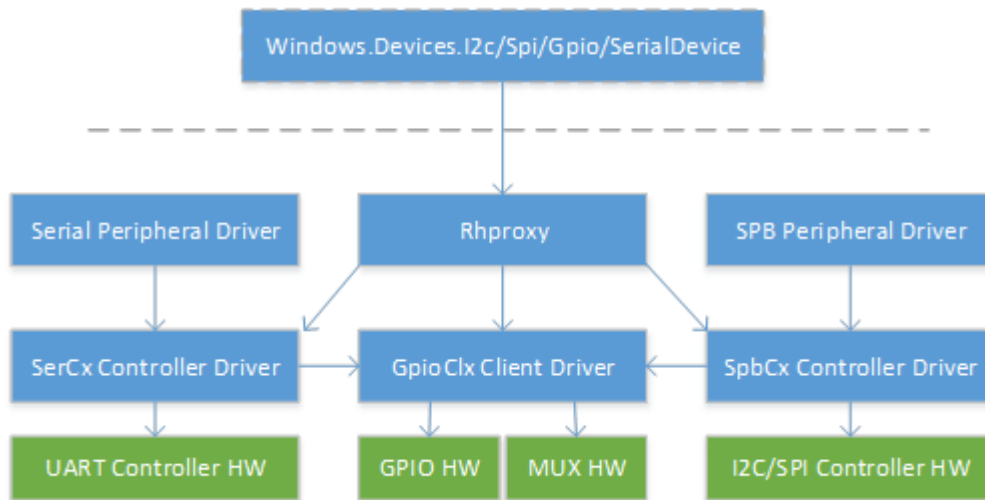
For example, assume that a pin's default muxing configuration is UART, and the pin can also be used as GPIO. When CLIENT_ConnectIoPins is called to connect the pin for GPIO, it should mux the pin to GPIO, and in CLIENT_DisconnectIoPins, it should mux the pin back to UART. In general, the Disconnect routines should undo operations done by the Connect routines.

Supporting muxing in SpbCx and SerCx controller drivers

As of Windows 10 build 14327, the SpbCx and SerCx frameworks contain built-in support for pin muxing that enables SpbCx and SerCx controller drivers to be pin muxing clients without any code changes to the controller drivers themselves. By

extension, any SpbCx/SerCx peripheral driver that connects to a muxing-enabled SpbCx/SerCx controller driver will trigger pin muxing activity.

The following diagram shows the dependencies between each of these components. As you can see, pin muxing introduces a dependency from SerCx and SpbCx controller drivers to the GPIO driver, which is usually responsible for muxing.



At device initialization time, the `SpbCx` and `SerCx` frameworks parse all `MsftFunctionConfig()` resources supplied as hardware resources to the device. `SpbCx/SerCx` then acquire and release the pin muxing resources on demand.

`SpbCx` applies pin muxing configuration in its `IRP_MJ_CREATE` handler, just before calling the client driver's `EvtSpbTargetConnect()` callback. If muxing configuration could not be applied, the controller driver's `EvtSpbTargetConnect()` callback will not be called. Therefore, an SPB controller driver may assume that pins are muxed to the SPB function by the time `EvtSpbTargetConnect()` is called.

`SpbCx` reverts pin muxing configuration in its `IRP_MJ_CLOSE` handler, just after invoking the controller driver's `EvtSpbTargetDisconnect()` callback. The result is that pins are muxed to the SPB function whenever a peripheral driver opens a handle to the SPB controller driver, and are muxed away when the peripheral driver closes their handle.

`SerCx` behaves similarly. `SerCx` acquires all `MsftFunctionConfig()` resources in its `IRP_MJ_CREATE` handler just before invoking the controller driver's `EvtSerCx2FileOpen()` callback, and releases all resources in its `IRP_MJ_CLOSE` handler, just after invoking the controller driver's `EvtSerCx2FileClose` callback.

The implication of dynamic pin muxing for `SerCx` and `SpbCx` controller drivers is that they must be able to tolerate pins being muxed away from SPB/UART function at certain times. Controller drivers need to assume that pins will not be muxed until `EvtSpbTargetConnect()` or `EvtSerCx2FileOpen()` is called. Pins are not necessary muxed

to SPB/UART function during the following callbacks. The following is not a complete list, but represents the most common PNP routines implemented by controller drivers.

- DriverEntry
- EvtDriverDeviceAdd
- EvtDevicePrepareHardware/EvtDeviceReleaseHardware
- EvtDeviceD0Entry/EvtDeviceD0Exit

Verification

When you're ready to test rhproxy, it's helpful to use the following step-by-step procedure.

1. Verify that each `SpbCx`, `GpioClx`, and `SerCx` controller driver is loading and operating correctly
2. Verify that `rhproxy` is present on the system. Some editions and builds of Windows do not have it.
3. Compile and load your rhproxy node using `ACPITABL.dat`
4. Verify that the `rhproxy` device node exists
5. Verify that `rhproxy` is loading and starting
6. Verify that the expected devices are exposed to user mode
7. Verify that you can interact with each device from the command line
8. Verify that you can interact with each device from a UWP app
9. Run HLK tests

Verify controller drivers

Since rhproxy exposes other devices on the system to user mode, it only works if those devices are already working. The first step is to verify that those devices - the I2C, SPI, GPIO controllers you wish to expose - are already working.

At the command prompt, run

```
ps
```

```
devcon status *
```

Look at the output and verify that all devices of interest are started. If a device has a problem code, you need to troubleshoot why that device is not loading. All devices should have been enabled during initial platform bringup. Troubleshooting `SpbCx`, `GpioClx`, or `SerCx` controller drivers is beyond the scope of this document.

Verify that rhpoxxy is present on the system

Verify that the `rhpoxxy` service is present on the system.

```
ps
```

```
reg query HKEY_LOCAL_MACHINE\SYSTEM\ControlSet001\Services\rhpoxxy
```

If the reg key is not present, rhpoxxy doesn't exist on your system. Rhpoxxy is present on all builds of IoT Core and Windows Enterprise build 15063 and later.

Compile and load ASL with ACPITABL.dat

Now that you've authored an rhpoxxy ASL node, it's time to compile and load it. You can compile the rhpoxxy node into a standalone AML file that can be appended to the system ACPI tables. Alternatively, if you have access to your system's ACPI sources, you can insert the rhpoxxy node directly to your platform's ACPI tables. However, during initial bringup it may be easier to use `ACPITABL.dat`.

1. Create a file named `yourboard.asl` and put the RHPX device node inside a `DefinitionBlock`:

```
C++
```

```
DefinitionBlock ("ACPITABL.dat", "SSDT", 1, "MSFT", "RHPROXY", 1)
{
    Scope (\_SB)
    {
        Device(RHPX)
        {
            ...
        }
    }
}
```

2. Download the [WDK](#) and find `asl.exe` at `C:\Program Files (x86)\Windows Kits\10\Tools\x64\ACPIVerify`
3. Run the following command to generate `ACPITABL.dat`:

```
ps
```

```
asl.exe yourboard.asl
```

4. Copy the resulting ACPITABL.dat file to c:\windows\system32 on your system under test.
5. Turn on test signing on your system under test:

```
ps  
  
bcdedit /set testsigning on
```

6. Reboot the system under test. The system will append the ACPI tables defined in ACPITABL.dat to the system firmware tables.

Verify that the rhproxy device node exists

Run the following command to enumerate the rhproxy device node.

```
ps  
  
devcon status *msft8000
```

The output of devcon should indicate that the device is present. If the device node is not present, the ACPI tables were not successfully added to the system.

Verify that rhproxy is loading and starting

Check the status of rhproxy:

```
ps  
  
devcon status *msft8000
```

If the output indicates that rhproxy is started, rhproxy has loaded and started successfully. If you see a problem code, you need to investigate. Some common problem codes are:

- Problem 51 - `CM_PROB_WAITING_ON_DEPENDENCY` - The system is not starting rhproxy because one of its dependencies has failed to load. This means that either the resources passed to rhproxy point to invalid ACPI nodes, or the target devices are not starting. First, double check that all devices are running successfully (see 'Verify controller drivers' above). Then, double check your ASL and ensure that all your resource paths (for example, `\_SB.I2C1`) are correct and point to valid nodes in your DSDT.

- Problem 10 - `CM_PROB_FAILED_START` - Rhproxy failed to start, most likely because of a resource parsing issue. Go over your ASL and double check resource indices in the DSD, and verify that GPIO resources are specified in increasing pin number order.

Verify that the expected devices are exposed to user mode

Now that rhproxy is running, it should have created devices interfaces that can be accessed by user mode. We will use several command line tools to enumerate devices and see that they're present.

Clone the <https://github.com/ms-iot/samples> repository and build the `GpioTestTool`, `I2cTestTool`, `SpiTestTool`, and `Mincomm` samples. Copy the tools to your device under test and use the following commands to enumerate devices.

```
ps
```

```
I2cTestTool.exe -list  
SpiTestTool.exe -list  
GpioTestTool.exe -list  
MinComm.exe -list
```

You should see your devices and friendly names listed. If you don't see the right devices and friendly names, double check your ASL.

Verify each device on the command line

The next step is to use the command line tools to open and interact with the devices.

I2CTestTool example:

```
ps
```

```
I2cTestTool.exe 0x55 I2C1  
> write {1 2 3}  
> read 3  
> writeread {1 2 3} 3
```

SpiTestTool example:

```
ps
```

```
SpiTestTool.exe -n SPI1
> write {1 2 3}
> read 3
```

GpioTestTool example:

```
ps

GpioTestTool.exe 12
> setdrivemode output
> write 0
> write 1
> setdrivemode input
> read
> interrupt on
> interrupt off
```

MinComm (serial) example. Connect Rx to Tx before running:

```
ps

MinComm "\\.\?ACPI#FSCL0007#3#{86e0d1e0-8089-11d0-9ce4-08003e301f73}\0000000000000006"
(type characters and see them echoed back)
```

Verify each device from a UWP app

Use the following samples to validate that devices work from UWP.

- [IoT-GPIO](#) 
- [IoT-I2C](#) 
- [IoT-SPI](#) 
- [CustomSerialDeviceAccess](#) 

Run the HLK Tests

Download the [Hardware Lab Kit \(HLK\)](#). The following tests are available:

- [GPIO WinRT Functional and Stress Tests](#)
- [I2C WinRT Write Tests \(EEPROM Required\)](#)
- [I2C WinRT Read Tests \(EEPROM Required\)](#)
- [I2C WinRT Nonexistent Slave Address Tests](#)
- [I2C WinRT Advanced Functional Tests \(mbed LPC1768 Required\)](#)
- [SPI WinRT Clock Frequency Verification Tests \(mbed LPC1768 Required\)](#)

- [SPI WinRT IO Transfer Tests \(mbed LPC1768 Required\)](#)
- [SPI WinRT Stride Verification Tests](#)
- [SPI WinRT Transfer Gap Detection Tests \(mbed LPC1768 Required\)](#)

When you select the rhproxy device node in HLK manager, the applicable tests will automatically be selected.

In the HLK manager, select "Resource Hub Proxy device":

The screenshot shows the Windows Hardware Lab Kit (HLK) manager interface. The title bar reads "Windows Hardware Lab Kit" with "Help | Configuration | Connect" on the right. The "mbm" logo is in the top left. Below the logo are tabs: "Project", "Selection", "Tests", "Results", and "Package". The "Selection" tab is active. Below the tabs are two dropdown menus: "\$\mypool" and "All", followed by a search bar. On the left, there are checkboxes for "show selected systems", "show selected devices and printers", "device manager", and "software device". Below these are "show" checkboxes for "Inbox" and "Hidden". The main area is a table with columns: "Name", "Platform", "Machine", and "Group". The "Resource Hub proxy device" is selected, highlighted in blue, and its group is "[Group 2]". Below the table, the device details are shown: "Mfg: Microsoft", "Device Name: Resource Hub proxy device", "Driver Name: c:\windows\system32\drivers\rhproxy.sys", and "Device Instance ID: ACPI\MSFT8000\1".

Name	Platform	Machine	Group
☐ PCI-to-PCI Bridge	Windows Mobile v10	jordanrh-mbm	
☐ Plug and Play Software Device Enumerator	Windows Mobile v10	jordanrh-mbm	
☐ Power Button	Windows Mobile v10	jordanrh-mbm	
☐ Power Button	Windows Mobile v10	jordanrh-mbm	
☐ Programmable interrupt controller	Windows Mobile v10	jordanrh-mbm	
☐ Realtek PCIe GBE Family Controller	Windows Mobile v10	jordanrh-mbm	
☒ Resource Hub proxy device	Windows Mobile v10	jordanrh-mbm	[Group 2]
☐ SD Storage Class Controller	Windows Mobile v10	jordanrh-mbm	
☐ SDA Standard Compliant SD Host Controller	Windows Mobile v10	jordanrh-mbm	
☐ SM Bus Controller	Windows Mobile v10	jordanrh-mbm	
☐ SPI Controller	Windows Mobile v10	jordanrh-mbm	
☐ Standard Power Management Controller	Windows Mobile v10	jordanrh-mbm	
☐ System CMOS/real time clock	Windows Mobile v10	jordanrh-mbm	

Mfg: **Microsoft**
Device Name: **Resource Hub proxy device**
Driver Name: **c:\windows\system32\drivers\rhproxy.sys**
Device Instance ID: **ACPI\MSFT8000\1**

Then click the Tests tab, and select I2C WinRT, Gpio WinRT, and Spi WinRT tests.

Windows Hardware Lab Kit

Help | Configuration | Connect

mbm

Project

Selection

Tests

Results

Package

Run Selected

View Details

Load Playlist

Save Selected As Playlist

☐	Status	Test Name	Type	Le	Tz	M
☐		DF - PNP Surprise Remove Device Test (Development)		03	Re	joi
☐		DF - PNP Surprise Remove Device Test (Reliability)		08	Re	joi
☐		DF - Reboot restart with IO before and after (Reliability)		04	Re	joi
☐		DF - Reboot Restart with IO During (Development and)		03	Re	joi
☐		DF - Reboot Restart with IO During (Reliability)		05	Re	joi
☐		DF - Sleep and PNP (disable and enable) with IO Befc		05	Re	joi
☐		DF - Sleep and PNP (disable and enable) with IO Befc		45	Re	joi
☐		DF - Sleep with IO Before and After (Bring Up)		04	Re	joi
☐		DF - Sleep with IO During (Development and Integrat		05	Re	joi
☐		DF - Sleep with IO During (Reliability)		45	Re	joi
☒		GPIO WinRT Functional and Stress Tests		05	Re	joi
☒		I2C WinRT Advanced Functional Tests (mbed LPC176		15	Re	joi
☒		I2C WinRT IO Stress Tests (EEPROMs Required)		04	Re	joi
☒		I2C WinRT Nonexistent Slave Address Tests		05	Re	joi
☒		I2C WinRT Read Tests (EEPROM Required)		05	Re	joi
☒		I2C WinRT Write Tests (EEPROM Required)		05	Re	joi
☐		WDK Guardrail Analysis		02	Re	joi

mbm

Database Project

Targets

Resource Hub proxy device

OS Platforms

Windows Mobile v10.0 x86

Product Types

Test Status

Passed: 0 test(s)

Failed: 0 test(s)

Assessment Passed: 0 test(s)

Assessment Failed: 0 test(s)

Not Run: 49 test(s)

Running: 0 test(s)

Total: 49 test(s)

Machine Status

jordanrh-mbm

Click Run Selected. Further documentation on each test is available by right clicking on the test and clicking "Test Description."

Resources

- [ACPI 5.0 specification](#)
- [Asl.exe \(Microsoft ASL Compiler\)](#)
- [Windows.Devices.Gpio](#)
- [Windows.Devices.I2c](#)
- [Windows.Devices.Spi](#)
- [Windows.Devices.SerialCommunication](#)
- [Test Authoring and Execution Framework \(TAEF\)](#)
- [SpbCx](#)
- [GpioClx](#)
- [SerCx](#)
- [MITT I2C Tests](#)
- [GpioTestTool](#)
- [I2cTestTool](#)
- [SpiTestTool](#)
- [MinComm \(Serial\)](#)
- [Hardware Lab Kit \(HLK\)](#)

Appendix

Appendix A - Raspberry Pi ASL Listing

See also [Raspberry Pi 2 & 3 Pin Mappings](#)

C++

```
DefinitionBlock ("ACPITABL.dat", "SSDT", 1, "MSFT", "RHProxy", 1)
{
    Scope (\_SB)
    {
        //
        // RHProxy Device Node to enable WinRT API
        //
        Device(RHPX)
        {
            Name(_HID, "MSFT8000")
            Name(_CID, "MSFT8000")
            Name(_UID, 1)

            Name(_CRS, ResourceTemplate()
            {
                // Index 0
                SPISerialBus(
                    // SCKL - GPIO 11 - Pin 23
                    // MOSI - GPIO 10 - Pin 19
                    // MISO - GPIO 9 - Pin 21
                    // CE0 - GPIO 8 - Pin 24
                    0,
                    // Device selection (CE0)
                    PolarityLow,
                    // Device selection polarity
                    FourWireMode,
                    // wiremode
                    0,
                    // databit len: placeholder
                    ControllerInitiated,
                    // slave mode
                    0,
                    // connection speed: placeholder
                    ClockPolarityLow,
                    // clock polarity: placeholder
                    ClockPhaseFirst,
                    // clock phase: placeholder
                    "\\_SB.SPI0",
                    // ResourceSource: SPI bus
                    controller name
                    0,
                    // ResourceSourceIndex
                    // Resource usage
                    )
                    // Vendor Data

                // Index 1
                SPISerialBus(
                    // SCKL - GPIO 11 - Pin 23
                    // MOSI - GPIO 10 - Pin 19
                    // MISO - GPIO 9 - Pin 21
                    // CE1 - GPIO 7 - Pin 26
                    1,
                    // Device selection (CE1)
                    PolarityLow,
                    // Device selection polarity
                    FourWireMode,
                    // wiremode
                    0,
                    // databit len: placeholder
```

```

        ControllerInitiated, // slave mode
        0, // connection speed: placeholder
        ClockPolarityLow, // clock polarity: placeholder
        ClockPhaseFirst, // clock phase: placeholder
        "\\_SB.SPI0", // ResourceSource: SPI bus
controller name
        0, // ResourceSourceIndex
        // Resource usage
    ) // Vendor Data

    // Index 2
    SPISerialBus( // SCKL - GPIO 21 - Pin 40
        // MOSI - GPIO 20 - Pin 38
        // MISO - GPIO 19 - Pin 35
        // CE1 - GPIO 17 - Pin 11
        1, // Device selection (CE1)
        PolarityLow, // Device selection polarity
        FourWireMode, // wiremode
        0, // databit len: placeholder
        ControllerInitiated, // slave mode
        0, // connection speed: placeholder
        ClockPolarityLow, // clock polarity: placeholder
        ClockPhaseFirst, // clock phase: placeholder
        "\\_SB.SPI1", // ResourceSource: SPI bus
controller name
        0, // ResourceSourceIndex
        // Resource usage
    ) // Vendor Data

    // Index 3
    I2CSerialBus( // Pin 3 (GPIO2, SDA1), 5 (GPIO3,
SCL1)
        0xFFFF, // SlaveAddress: placeholder
        , // SlaveMode: default to
ControllerInitiated
        0, // ConnectionSpeed: placeholder
        , // Addressing Mode: placeholder
        "\\_SB.I2C1", // ResourceSource: I2C bus
controller name
        ,
        ,
    ) // VendorData

    // Index 4 - GPIO 4 -
    GpioIO(Shared, PullUp, , , , "\\_SB.GPIO", , , , ) { 4 }
    GpioInt(Edge, ActiveBoth, Shared, PullUp, 0, "\\_SB.GPIO",)
{ 4 }

    // Index 6 - GPIO 5 -
    GpioIO(Shared, PullUp, , , , "\\_SB.GPIO", , , , ) { 5 }
    GpioInt(Edge, ActiveBoth, Shared, PullUp, 0, "\\_SB.GPIO",)
{ 5 }

    // Index 8 - GPIO 6 -
    GpioIO(Shared, PullUp, , , , "\\_SB.GPIO", , , , ) { 6 }
    GpioInt(Edge, ActiveBoth, Shared, PullUp, 0, "\\_SB.GPIO",)
{ 6 }

    // Index 10 - GPIO 12 -

```

```

        GpioIO(Shared, PullDown, , , , "\\_SB.GPI0", , , , ) { 12 }
        GpioInt(Edge, ActiveBoth, Shared, PullDown, 0,
"\_SB.GPI0",) { 12 }
        // Index 12 - GPIO 13 -
        GpioIO(Shared, PullDown, , , , "\\_SB.GPI0", , , , ) { 13 }
        GpioInt(Edge, ActiveBoth, Shared, PullDown, 0,
"\_SB.GPI0",) { 13 }
        // Index 14 - GPIO 16 -
        GpioIO(Shared, PullDown, , , , "\\_SB.GPI0", , , , ) { 16 }
        GpioInt(Edge, ActiveBoth, Shared, PullDown, 0,
"\_SB.GPI0",) { 16 }
        // Index 16 - GPIO 18 -
        GpioIO(Shared, PullDown, , , , "\\_SB.GPI0", , , , ) { 18 }
        GpioInt(Edge, ActiveBoth, Shared, PullDown, 0,
"\_SB.GPI0",) { 18 }
        // Index 18 - GPIO 22 -
        GpioIO(Shared, PullDown, , , , "\\_SB.GPI0", , , , ) { 22 }
        GpioInt(Edge, ActiveBoth, Shared, PullDown, 0,
"\_SB.GPI0",) { 22 }
        // Index 20 - GPIO 23 -
        GpioIO(Shared, PullDown, , , , "\\_SB.GPI0", , , , ) { 23 }
        GpioInt(Edge, ActiveBoth, Shared, PullDown, 0,
"\_SB.GPI0",) { 23 }
        // Index 22 - GPIO 24 -
        GpioIO(Shared, PullDown, , , , "\\_SB.GPI0", , , , ) { 24 }
        GpioInt(Edge, ActiveBoth, Shared, PullDown, 0,
"\_SB.GPI0",) { 24 }
        // Index 24 - GPIO 25 -
        GpioIO(Shared, PullDown, , , , "\\_SB.GPI0", , , , ) { 25 }
        GpioInt(Edge, ActiveBoth, Shared, PullDown, 0,
"\_SB.GPI0",) { 25 }
        // Index 26 - GPIO 26 -
        GpioIO(Shared, PullDown, , , , "\\_SB.GPI0", , , , ) { 26 }
        GpioInt(Edge, ActiveBoth, Shared, PullDown, 0,
"\_SB.GPI0",) { 26 }
        // Index 28 - GPIO 27 -
        GpioIO(Shared, PullDown, , , , "\\_SB.GPI0", , , , ) { 27 }
        GpioInt(Edge, ActiveBoth, Shared, PullDown, 0,
"\_SB.GPI0",) { 27 }
        // Index 30 - GPIO 35 -
        GpioIO(Shared, PullUp, , , , "\\_SB.GPI0", , , , ) { 35 }
        GpioInt(Edge, ActiveBoth, Shared, PullUp, 0, "\\_SB.GPI0",)
{ 35 }

        // Index 32 - GPIO 47 -
        GpioIO(Shared, PullUp, , , , "\\_SB.GPI0", , , , ) { 47 }
        GpioInt(Edge, ActiveBoth, Shared, PullUp, 0, "\\_SB.GPI0",)
{ 47 }

    })

    Name(_DSD, Package()
    {
        ToUUID("daffd814-6eba-4d8c-8a91-bc9bbf4aa301"),
        Package()
        {
            // Reference

```

```

http://www.raspberrypi.org/documentation/hardware/raspberrypi/spi/README.md
// SPI 0
Package(2) { "bus-SPI-SPI0", Package() { 0, 1 }},
// Index 0 & 1
Package(2) { "SPI0-MinClockInHz", 7629 },
// 7629 Hz
Package(2) { "SPI0-MaxClockInHz", 125000000 },
// 125 MHz
Package(2) { "SPI0-SupportedDataBitLengths", Package() {
8 }}, // Data Bit Length
// SPI 1
Package(2) { "bus-SPI-SPI1", Package() { 2 }},
// Index 2
Package(2) { "SPI1-MinClockInHz", 30518 },
// 30518 Hz
Package(2) { "SPI1-MaxClockInHz", 125000000 },
// 125 MHz
Package(2) { "SPI1-SupportedDataBitLengths", Package() {
8 }}, // Data Bit Length
// I2C1
Package(2) { "bus-I2C-I2C1", Package() { 3 }},
// GPIO Pin Count and supported drive modes
Package (2) { "GPIO-PinCount", 54 },
Package (2) { "GPIO-UseDescriptorPinNumbers", 1 },
Package (2) { "GPIO-SupportedDriveModes", 0xf },
// InputHighImpedance, InputPullUp, InputPullDown, OutputCmos
    }
  })
}
}
}
}

```

Appendix B - MinnowBoardMax ASL Listing

See also [MinnowBoard Max Pin Mappings](#)

C++

```

DefinitionBlock ("ACPITABL.dat", "SSDT", 1, "MSFT", "RHPROXY", 1)
{
    Scope (\_SB)
    {
        Device(RHPX)
        {
            Name(_HID, "MSFT8000")
            Name(_CID, "MSFT8000")
            Name(_UID, 1)

            Name(_CRS, ResourceTemplate()
            {
                // Index 0
            }
        }
    }
}

```

```

        SPISerialBus(                                // Pin 5, 7, 9 , 11 of JP1 for
SIO_SPI
        1,                                            // Device selection
        PolarityLow,                                // Device selection polarity
        FourWireMode,                                // wiremode
        8,                                            // databit len
        ControllerInitiated,                        // slave mode
        8000000,                                    // Connection speed
        ClockPolarityLow,                            // Clock polarity
        ClockPhaseSecond,                            // clock phase
        "\\_SB.SPI1",                                // ResourceSource: SPI bus
controller name
        0,                                            // ResourceSourceIndex
        ResourceConsumer,                            // Resource usage
        JSPI,                                        // DescriptorName: creates name
for offset of resource descriptor
        )                                            // Vendor Data

        // Index 1
        I2CSerialBus(                                // Pin 13, 15 of JP1, for SIO_I2C5
(signal)
        0xFF,                                        // SlaveAddress: bus address
        ,                                            // SlaveMode: default to
ControllerInitiated
        400000,                                    // ConnectionSpeed: in Hz
        ,                                            // Addressing Mode: default to 7
bit
        "\\_SB.I2C6",                                // ResourceSource: I2C bus
controller name (For MinnowBoard Max, hardware I2C5(0-based) is reported as
ACPI I2C6(1-based))
        ,
        ,
        JI2C,                                        // Descriptor Name: creates name
for offset of resource descriptor
        )                                            // VendorData

        // Index 2
        UARTSerialBus(                                // Pin 17, 19 of JP1, for SIO_UART2
        115200,                                    // InitialBaudRate: in bits per
second
        ,                                            // BitsPerByte: default to 8 bits
        ,                                            // StopBits: Defaults to one bit
        0xfc,                                        // LinesInUse: 8 1-bit flags to
declare line enabled
        ,                                            // IsBigEndian: default to
LittleEndian
        ,                                            // Parity: Defaults to no parity
        ,                                            // FlowControl: Defaults to no
flow control
        32,                                        // ReceiveBufferSize
        32,                                        // TransmitBufferSize
        "\\_SB.URT2",                                // ResourceSource: UART bus
controller name
        ,
        ,

```

```

        UAR2,                // DescriptorName: creates name
for offset of resource descriptor
    )

    // Index 3
    GpioIo (Shared, PullNone, 0, 0, IoRestrictionNone,
"\_SB.GP02",) {0} // Pin 21 of JP1 (GPIO_S5[00])
    // Index 4
    GpioInt(Edge, ActiveBoth, SharedAndWake, PullNone,
0, "\_SB.GP02",) {0}

    // Index 5
    GpioIo (Shared, PullNone, 0, 0, IoRestrictionNone,
"\_SB.GP02",) {1} // Pin 23 of JP1 (GPIO_S5[01])
    // Index 6
    GpioInt(Edge, ActiveBoth, SharedAndWake, PullNone,
0, "\_SB.GP02",) {1}

    // Index 7
    GpioIo (Shared, PullNone, 0, 0, IoRestrictionNone,
"\_SB.GP02",) {2} // Pin 25 of JP1 (GPIO_S5[02])
    // Index 8
    GpioInt(Edge, ActiveBoth, SharedAndWake, PullNone,
0, "\_SB.GP02",) {2}

    // Index 9
    UARTSerialBus(           // Pin 6, 8, 10, 12 of JP1, for
SIO_UART1
        115200,              // InitialBaudRate: in bits per
second
        ,                    // BitsPerByte: default to 8 bits
        ,                    // StopBits: Defaults to one bit
        0xfc,                // LinesInUse: 8 1-bit flags to
declare line enabled
        ,                    // IsBigEndian: default to
LittleEndian
        ,                    // Parity: Defaults to no parity
        FlowControlHardware, // FlowControl: Defaults to no
flow control
        32,                  // ReceiveBufferSize
        32,                  // TransmitBufferSize
        "\_SB.URT1",         // ResourceSource: UART bus
controller name
        ,
        ,
        UAR1,                // DescriptorName: creates name for
offset of resource descriptor
    )

    // Index 10
    GpioIo (Shared, PullNone, 0, 0, IoRestrictionNone,
"\_SB.GP00",) {62} // Pin 14 of JP1 (GPIO_SC[62])
    // Index 11
    GpioInt(Edge, ActiveBoth, SharedAndWake, PullNone,
0, "\_SB.GP00",) {62}

```

```

        // Index 12
        GpioIo (Shared, PullNone, 0, 0, IoRestrictionNone,
"\_SB.GP00",) {63} // Pin 16 of JP1 (GPIO_SC[63])
        // Index 13
        GpioInt(Edge, ActiveBoth, SharedAndWake, PullNone,
0, "\_SB.GP00",) {63}

        // Index 14
        GpioIo (Shared, PullNone, 0, 0, IoRestrictionNone,
"\_SB.GP00",) {65} // Pin 18 of JP1 (GPIO_SC[65])
        // Index 15
        GpioInt(Edge, ActiveBoth, SharedAndWake, PullNone,
0, "\_SB.GP00",) {65}

        // Index 16
        GpioIo (Shared, PullNone, 0, 0, IoRestrictionNone,
"\_SB.GP00",) {64} // Pin 20 of JP1 (GPIO_SC[64])
        // Index 17
        GpioInt(Edge, ActiveBoth, SharedAndWake, PullNone,
0, "\_SB.GP00",) {64}

        // Index 18
        GpioIo (Shared, PullNone, 0, 0, IoRestrictionNone,
"\_SB.GP00",) {94} // Pin 22 of JP1 (GPIO_SC[94])
        // Index 19
        GpioInt(Edge, ActiveBoth, SharedAndWake, PullNone,
0, "\_SB.GP00",) {94}

        // Index 20
        GpioIo (Shared, PullNone, 0, 0, IoRestrictionNone,
"\_SB.GP00",) {95} // Pin 24 of JP1 (GPIO_SC[95])
        // Index 21
        GpioInt(Edge, ActiveBoth, SharedAndWake, PullNone,
0, "\_SB.GP00",) {95}

        // Index 22
        GpioIo (Shared, PullNone, 0, 0, IoRestrictionNone,
"\_SB.GP00",) {54} // Pin 26 of JP1 (GPIO_SC[54])
        // Index 23
        GpioInt(Edge, ActiveBoth, SharedAndWake, PullNone,
0, "\_SB.GP00",) {54}
    })

    Name(_DSD, Package()
    {
        ToUUID("daffd814-6eba-4d8c-8a91-bc9bbf4aa301"),
        Package()
        {
            // SPI Mapping
            Package(2) { "bus-SPI-SPI0", Package() { 0 } },

            Package(2) { "SPI0-MinClockInHz", 100000 },
            Package(2) { "SPI0-MaxClockInHz", 15000000 },
            // SupportedDataBitLengths takes a list of support data

```



```
$(($_.PinNumber) }  
"@  
    Write-Host $a  
    $resourceIndex += 2;  
}
```

Enumerate devices

Article • 05/06/2023

The [Windows.Devices Enumeration](#) APIs let you find devices that are internally connected to the system, externally connected, or detectable over wireless or networking protocols.

Samples

The easiest way to enumerate all available devices is to take a snapshot with the [FindAllAsync](#) command (explained further in a section below).

CSharp

```
async void enumerateSnapshot(){
    DeviceInformationCollection collection = await
    DeviceInformation.FindAllAsync();
}
```

See the [Device enumeration and pairing sample](#) [↗](#) for a more advanced example of the [Windows.Devices Enumeration](#) APIs.

Enumeration APIs

The [Windows.Devices Enumeration](#) namespace enables you to find devices that are internally connected to the system, externally connected, or detectable over wireless or networking protocols. Some of the features supported by these APIs include:

- Finding a device to connect to with your application.
- Getting information about devices connected to or discoverable by the system.
- Have an app receive notifications when devices are added, connect, disconnect, change online status, or change other properties.
- Have an app receive background triggers when devices connect, disconnect, change online status, or change other properties.

These APIs can enumerate devices over any of the following protocols and buses, provided the individual device and the system running the app support that technology. This is not an exhaustive list, and other protocols may be supported by a specific device.

- Physically connected buses. This includes PCI and USB. For example, anything that you can see in the **Device Manager**.

- [UPnP](#)
- Digital Living Network Alliance (DLNA)
- [Discovery and Launch \(DIAL\)](#)
- [DNS Service Discovery \(DNS-SD\)](#)
- [Web Services on Devices \(WSD\)](#)
- [Bluetooth](#)
- [Wi-Fi Direct](#)
- [WiGig](#)
- [Point of Service](#)

In many cases, you will not need to worry about using the enumeration APIs. This is because many APIs that use devices will automatically select the appropriate default device or provide a more streamlined enumeration API. For example, [MediaElement](#) will automatically use the default audio renderer device. As long as your app can use the default device, there is no need to use the enumeration APIs in your application. The enumeration APIs provide a general and flexible way for you to discover and connect to available devices. This topic provides information about enumerating devices and describes the four common ways to enumerate devices.

- Using the [DevicePicker](#) UI
- Enumerating a snapshot of devices currently discoverable by the system
- Enumerating devices currently discoverable and watch for changes
- Enumerating devices currently discoverable and watch for changes in a background task

DeviceInformation objects

Working with the enumeration APIs, you will frequently need to use [DeviceInformation](#) objects. These objects contain most of the available information about the device. The following table explains some of the **DeviceInformation** properties you will be interested in. For a complete list, see the reference page for **DeviceInformation**.

Property	Comments
DeviceInformation.Id	This is the unique identifier of the device and is provided as a string variable. In most cases, this is an opaque value you will just pass from one method to another to indicate the specific device you are interested in. You can also use this property and the DeviceInformation.Kind property after closing down your app and reopening it. This will ensure that you can recover and reuse the same DeviceInformation object.

Property	Comments
DeviceInformation.Kind	This indicates the kind of device object represented by the DeviceInformation object. This is not the device category or type of device. A single device can be represented by several different DeviceInformation objects of different kinds. The possible values for this property are listed in DeviceInformationKind as well as how they relate to one another.
DeviceInformation.Properties	This property bag contains information that is requested for the DeviceInformation object. The most common properties are easily referenced as properties of the DeviceInformation object, such as with DeviceInformation.Name . For more information, see Device information properties .

DevicePicker user interface

The [DevicePicker](#) is a control provided by Windows that creates a small UI that enables the user to select a device from a list. It can be customized in a few ways, such as:

- Control devices that are displayed in the UI by adding a [SupportedDeviceSelectors](#), a [SupportedDeviceClasses](#), or both to the [DevicePicker.Filter](#). In most cases, you only need to add one selector or class, but if you do need more than one you can add multiple. If you do add multiple selectors or classes, they are conjoined using an OR logic function.
- Specify the properties you want to retrieve for the devices. You can do this by adding properties to [DevicePicker.RequestedProperties](#).
- Alter the appearance of the [DevicePicker](#) using [Appearance](#).
- Specify the size and location of the [DevicePicker](#) when it is displayed.

While the [DevicePicker](#) is displayed, the contents of the UI will be automatically updated if devices are added, removed, or updated.

Note You cannot specify the [DeviceInformationKind](#) using the [DevicePicker](#). If you want to have devices of a specific **DeviceInformationKind**, you will need to build a [DeviceWatcher](#) and provide your own UI.

Casting media content and DIAL also each provide their own pickers if you want to use them. They are [CastingDevicePicker](#) and [DialDevicePicker](#), respectively.

Enumerate a snapshot of devices

In some scenarios, the [DevicePicker](#) will not be suitable for your needs and you need something more flexible. Perhaps you want to build your own UI or need to enumerate devices without displaying a UI to the user. In these situations, you could enumerate a snapshot of devices. This involves looking through the devices that are currently connected to or paired with the system. However, you need to be aware that this method only looks at a snapshot of devices that are available, so you will not be able to find devices that connect after you enumerate through the list. You also will not be notified if a device is updated or removed. Another potential downside to be aware of is that this method will hold back any results until the entire enumeration is completed. For this reason, you should not use this method when you are interested in **AssociationEndpoint**, **AssociationEndpointContainer**, or **AssociationEndpointService** objects since they are found over a network or wireless protocol. This can take up to 30 seconds to complete. In that scenario, you should use a [DeviceWatcher](#) object to enumerate through the possible devices.

To enumerate through a snapshot of devices, use the [FindAllAsync](#) method. This method waits until the entire enumeration process is complete and returns all the results as one [DeviceInformationCollection](#) object. This method is also overloaded to provide you with several options for filtering your results and limiting them to the devices that you are interested in. You can do this by providing a [DeviceClass](#) or passing in a device selector. The device selector is an Advanced Query Syntax (AQS) string that specifies the devices you want to enumerate. For more information, see [Build a device selector](#).

In addition to limiting the results, you can also specify the properties that you want to retrieve for the devices. If you do, the specified properties will be available in the property bag for each of the [DeviceInformation](#) objects returned in the collection. It is important to note that not all properties are available for all device kinds. To see what properties are available for which device kinds, see [Device information properties](#).

Enumerate and watch devices

A more powerful and flexible method of enumerating devices is creating a [DeviceWatcher](#). This option provides the most flexibility when you are enumerating devices. It allows you to enumerate devices that are currently present, and also receive notifications when devices that match your device selector are added, removed, or properties change. When you create a **DeviceWatcher**, you provide a device selector. For more information about device selectors, see [Build a device selector](#). After creating

the watcher, you will receive the following notifications for any device that matches your provided criteria.

- Add notification when a new device is added.
- Update notification when a property you are interested in is changed.
- Remove notification when a device is no longer available or no longer matches your filter.

In most cases where you are using a [DeviceWatcher](#), you are maintaining a list of devices and adding to it, removing items from it, or updating items as your watcher receives updates from the devices that you are watching. When you receive an update notification, the updated information will be available as a [DeviceInformationUpdate](#) object. In order to update your list of devices, first find the appropriate [DeviceInformation](#) that changed. Then call the [Update](#) method for that object, providing the [DeviceInformationUpdate](#) object. This is a convenience function that will automatically update your [DeviceInformation](#) object.

Since a [DeviceWatcher](#) sends notifications as devices arrive and when they change, you should use this method of enumerating devices when you are interested in [AssociationEndpoint](#), [AssociationEndpointContainer](#), or [AssociationEndpointService](#) objects since they are enumerated over networking or wireless protocols.

To create a [DeviceWatcher](#), use one of the [CreateWatcher](#) methods. These methods are overloaded to enable you to specify the devices that you are interested in. You can do this by providing a [DeviceClass](#) or passing in a device selector. The device selector is an AQS string that specifies the devices you want to enumerate. For more information, see [Build a device selector](#). You can also specify the properties that you want to retrieve for the devices and are interested in. If you do, the specified properties will be available in the property bag for each of the [DeviceInformation](#) objects returned in the collection. It is important to note that not all properties are available for all device kinds. To see what properties are available for which device kinds, see [Device information properties](#)

Watch devices as a background task

Watching devices as a background task is very similar to creating a [DeviceWatcher](#) as described above. In fact, you will still need to create a normal [DeviceWatcher](#) object first as described in the previous section. Once you create it, you call [GetBackgroundTrigger](#) instead of [DeviceWatcher.Start](#). When you call [GetBackgroundTrigger](#), you must specify which of the notifications you are interested in: add, remove, or update. You cannot request update or remove without requesting add as well. Once you register the trigger, the [DeviceWatcher](#) will start running immediately in the background. From this point forward, whenever it receives a new notification for your application that matches your

criteria, the background task will trigger and it will provide you the latest changes since it last triggered your application.

Important The first time that a [DeviceWatcherTrigger](#) triggers your application will be when the watcher reaches the **EnumerationCompleted** state. This means it will contain all of the initial results. Any future times it triggers your application, it will only contain the add, update, and remove notifications that have occurred since the last trigger. This is slightly different from a foreground [DeviceWatcher](#) object because the initial results do not come in one at a time and are only delivered in a bundle after the **EnumerationCompleted** is reached.

Some wireless protocols behave differently if they are scanning in the background versus the foreground, or they may not support scanning in the background at all. There are three possibilities with relation to background scanning. The following table lists the possibilities and the effects this may have on your application. For example, Bluetooth and Wi-Fi Direct do not support background scans, so by extension, they do not support a [DeviceWatcherTrigger](#).

Behavior	Impact
Same behavior in background	None
Only passive scans possible in background	Device may take longer to discover while waiting for a passive scan to occur.
Background scans not supported	No devices will be detectable by the DeviceWatcherTrigger , and no updates will be reported.

If your [DeviceWatcherTrigger](#) includes a protocol that does not support scanning in as a background task, your trigger will still work. However, you will not be able to get any updates or results over that protocol. The updates for other protocols or devices will still be detected normally.

Using DeviceInformationKind

In most scenarios, you will not need to worry about the [DeviceInformationKind](#) of a [DeviceInformation](#) object. This is because the device selector returned by the device API you're using will often guarantee you are getting the correct kinds of device objects to use with their API. However, in some scenarios you will want to get the

DeviceInformation for devices, but there is not a corresponding device API to provide a device selector. In these cases you will need to build your own selector. For example, Web Services on Devices does not have a dedicated API, but you can discover those devices and get information about them using the [Windows.Devices.Enumeration](#) APIs and then use them using the socket APIs.

If you are building your own device selector to enumerate through device objects, [DeviceInformationKind](#) will be important for you to understand. All of the possible kinds, as well as how they relate to one another, are described on the reference page for **DeviceInformationKind**. One of the most common uses of **DeviceInformationKind** is to specify what kind of devices you are searching for when submitting a query in conjunction with a device selector. By doing this, it makes sure that you only enumerate over devices that match the provided **DeviceInformationKind**. For example, you could find a **DeviceInterface** object and then run a query to get the information for the parent **Device** object. That parent object may contain additional information.

It is important to note that the properties available in the property bag for a **DeviceInformation** object will vary depending on the [DeviceInformationKind](#) of the device. Certain properties are only available with certain kinds. For more information about which properties are available for which kinds, see [Device information properties](#). Hence, in the above example, searching for the parent **Device** will give you access to more information that was not available from the **DeviceInterface** device object. Because of this, when you create your AQS filter strings, it is important to ensure that the requested properties are available for the **DeviceInformationKind** objects you are enumerating. For more information about building a filter, see [Build a device selector](#).

When enumerating **AssociationEndpoint**, **AssociationEndpointContainer**, or **AssociationEndpointService** objects, you are enumerating over a wireless or network protocol. In these situations, we recommend that you don't use [FindAllAsync](#) and instead use [CreateWatcher](#). This is because searching over a network often results in search operations that won't timeout for 10 or more seconds before generating [EnumerationCompleted](#). [FindAllAsync](#) doesn't complete its operation until **EnumerationCompleted** is triggered. If you are using a [DeviceWatcher](#), you'll get results closer to real time regardless of when **EnumerationCompleted** is called.

Save a device for later use

Any [DeviceInformation](#) object is uniquely identified by a combination of two pieces of information: [DeviceInformation.Id](#) and [DeviceInformation.Kind](#). If you keep these two pieces of information, you can recreate a **DeviceInformation** object after it is lost by

supplying this information to [CreateFromIdAsync](#). If you do this, you can save user preferences for a device that integrates with your app.

Build a device selector

Article • 05/06/2023

A device selector lets you limit the devices to search through when enumerating devices, which helps you get more relevant results and improve the performance of the system.

In most scenarios you get a device selector from a device stack. For example, you might use [GetDeviceSelector](#) for devices discovered over USB. These device selectors return an Advanced Query Syntax (AQS) string. For more details on the AQS format, see [Using Advanced Query Syntax Programmatically](#).

Important APIs

- [Windows.Devices.Enumeration](#)

Building the filter string

There are some cases where you need to enumerate devices and a provided device selector is not available for your scenario. A device selector is an AQS filter string that contains the following information. Before creating a filter string, you need to know some key pieces of information about the devices you want to enumerate.

- The [DeviceInformationKind](#) of the devices you are interested in. For more information about how [DeviceInformationKind](#) impacts enumerating devices, see [Enumerate devices](#).
- How to build an AQS filter string (explained in this topic).
- The properties you are interested in. The available properties will depend upon the [DeviceInformationKind](#). See [Device information properties](#) for more information.
- The protocols you are querying over. This is only needed if you are searching for devices over a wireless or wired network. For more information about doing this, see [Enumerate devices over a network](#).

When using the [Windows.Devices.Enumeration](#) APIs, you frequently combine the device selector with the device kind that you are interested in. The available list of device kinds is defined by the [DeviceInformationKind](#) enumeration. This combination of factors helps you to limit the devices that are available to the ones that you are interested in. If you do not specify the [DeviceInformationKind](#), or the method you are using does not provide a [DeviceInformationKind](#) parameter, the default kind is [DeviceInterface](#).

The [Windows.Devices.Enumeration](#) APIs use canonical AQS syntax, but not all of the operators are supported. For a list of properties that are available when you are constructing your filter string, see [Device information properties](#).

⊗ **Caution**

Custom properties that are defined using the `{GUID} PID` format cannot be used when constructing your AQS filter string. This is because the property type is derived from the well-known property name.

The following table lists the AQS operators and what types of parameters they support.

Operator	Supported types
COP_EQUAL	String, boolean, GUID, UInt16, UInt32
COP_NOTEQUAL	String, boolean, GUID, UInt16, UInt32
COP_LESSTHAN	UInt16, UInt32
COP_GREATERTHAN	UInt16, UInt32
COP_LESSTHANOEQUAL	UInt16, UInt32
COP_GREATERTHANOEQUAL	UInt16, UInt32
COP_VALUE_CONTAINS	String, string array, boolean array, GUID array, UInt16 array, UInt32 array
COP_VALUE_NOTCONTAINS	String, string array, boolean array, GUID array, UInt16 array, UInt32 array
COP_VALUE_STARTSWITH	String
COP_VALUE_ENDSWITH	String
COP_DOSWILDCARDS	Not supported
COP_WORD_EQUAL	Not supported
COP_WORD_STARTSWITH	Not supported
COP_APPLICATION_SPECIFIC	Not supported

You can specify **NULL** for **COP_EQUAL** or **COP_NOTEQUAL**. This translates to a property with no value, or that the value does not exist. In AQS, you specify **NULL** by using empty brackets `[]`.

Important

When using the **COP_VALUE_CONTAINS** and **COP_VALUE_NOTCONTAINS** operators, they behave differently with strings and string arrays. In the case of a string, the system will perform a case-insensitive search to see if the device contains the indicated string as a substring. In the case of a string array, substrings are not searched. With the string array, the array is searched to see if it contains the entire specified string. It is not possible to search a string array to see if the elements in the array contain a substring.

If you cannot create a single AQS filter string that will scope your results appropriately, you can filter your results after you receive them. However, we recommend limiting the results from your initial AQS filter string as much as possible when you provide it to the [Windows.Devices.Enumeration](#) APIs. This will help improve the performance of your application.

AQS string examples

The following examples demonstrate how the AQS syntax can be used to limit the devices you want to enumerate. All of these filter strings are paired up with a [DeviceInformationKind](#) to create a complete filter. If no kind is specified, remember that the default kind is **DeviceInterface**.

When this filter is paired with a [DeviceInformationKind](#) of **DeviceInterface**, it enumerates all objects that contain the Audio Capture interface class and that are currently enabled. = translates to **COP_EQUALS**.

syntax

```
System.Devices.InterfaceClassGuid:="{2eef81be-33fa-4800-9670-1cd474972c3f}"  
AND  
System.Devices.InterfaceEnabled:=System.StructuredQueryType.Boolean#True
```

When this filter is paired with a [DeviceInformationKind](#) of **Device**, it enumerates all objects that have at least one hardware id of GenCdRom. ~~ translates to **COP_VALUE_CONTAINS**.

syntax

```
System.Devices.HardwareIds:~~"GenCdRom"
```

When this filter is paired with a [DeviceInformationKind](#) of **DeviceContainer**, it enumerates all objects that have a model name containing the substring Microsoft. ~ ~ translates to **COP_VALUE_CONTAINS**.

syntax

```
System.Devices.ModelName:~~"Microsoft"
```

When this filter is paired with a [DeviceInformationKind](#) of **DeviceInterface**, it enumerates all objects that have a name starting with the substring Microsoft. ~ < translates to **COP_STARTSWITH**.

syntax

```
System.ItemNameDisplay:~<"Microsoft"
```

When this filter is paired with a [DeviceInformationKind](#) of **Device**, it enumerates all objects that have a **System.Devices.IpAddress** property set. <>[] translates to **COP_NOTEQUALS** combined with a **NULL** value.

syntax

```
System.Devices.IpAddress:<>[]
```

When this filter is paired with a [DeviceInformationKind](#) of **Device**, it enumerates all objects that do not have a **System.Devices.IpAddress** property set. =[] translates to **COP_EQUALS** combined with a **NULL** value.

syntax

```
System.Devices.IpAddress:=[]
```

See Also

- [Using Advanced Query Syntax Programmatically](#)
- [Windows.Devices.Enumeration](#)

Enumerate devices over a network

Article • 05/06/2023

Use the [Windows.Devices.Enumeration](#) APIs to enumerate devices over wireless and networked protocols.

Important APIs

- [Windows.Devices.Enumeration](#)

Enumerating devices over networked or wireless protocols

Sometimes you need to enumerate devices that are not locally connected and can only be discovered over a wireless or networking protocols. In order to do so, the [Windows.Devices.Enumeration](#) APIs have three different kinds of device objects: the **AssociationEndpoint** (AEP), the **AssociationEndpointContainer** (AEP Container), and the **AssociationEndpointService** (AEP Service). As a group these are referred to as AEPs or AEP objects.

Some device APIs provide a selector string that you can use to enumerate through the available AEP objects. This could include both devices that are paired and are not paired with the system. Some of the devices might not require pairing. Those device APIs may attempt to pair the device if pairing it is necessary before interacting with it. Wi-Fi Direct is an example of APIs that follow this pattern. If those device APIs do not automatically pair the device, you can pair it using the [DeviceInformationPairing](#) object available from [DeviceInformationPairing](#).

However, there may be cases where you want to manually discover devices on your own without using a pre-defined selector string. For example, you may just need to gather information about AEP devices without interacting with them or you may want to find more AEP objects than will be discovered with the pre-defined selector string. In this case, you will build your own selector string and use it following the instructions under [Build a device selector](#).

When you build your own selector, it is strongly recommended that you limit your scope of enumeration to the protocols that you are interested in. For example, you don't want to have the Wi-Fi radio search for Wi-Fi Direct devices if you are particularly interested in UPnP devices. Windows has defined an identity for each protocol that you can use to scope your enumeration. The following table lists the protocol types and identifiers.

Protocol or network device type	Id
UPnP (including DIAL and DLNA)	{0e261de4-12f0-46e6-91ba-428607ccef64}
Web services on devices (WSD)	{782232aa-a2f9-4993-971b-aedc551346b0}
Wi-Fi Direct	{0407d24e-53de-4c9a-9ba1-9ced54641188}
DNS service discovery (DNS-SD)	{4526e8c1-8aac-4153-9b16-55e86ada0e54}
Point of service	{d4bf61b3-442e-4ada-882d-fa7B70c832d9}
Network printers (active directory printers)	{37aba761-2124-454c-8d82-c42962c2de2b}
Windows connect now (WNC)	{4c1b1ef8-2f62-4b9f-9bc5-b21ab636138f}
WiGig docks	{a277f3a5-8764-4f88-8045-4c5e962640b1}
Wi-Fi provisioning for HP printers	{c85ef710-f344-4792-bb6d-85a4346f1e69}
Bluetooth	{e0cbf06c-cd8b-4647-bb8a-263b43f0f974}
Bluetooth LE	{bb7bb05e-5972-42b5-94fc-76eaa7084d49}
Network Camera	{b8238652-b500-41eb-b4f3-4234f7f5ae99}

Advanced Query Syntax (AQS) examples

Each AEP kind has a property you can use to constrain your enumeration to a specific protocol. Keep in mind you can use the OR operator in an AQS filter to combine multiple protocols. Here are some examples of AQS filter strings that show how to query for AEP devices.

This AQS queries for all UPnP **AssociationEndpoint** objects when the **DeviceInformationKind** is set to **AssocationEndpoint**.

syntax

```
System.Devices.Aep.ProtocolId="{0e261de4-12f0-46e6-91ba-428607ccef64}"
```

This AQS queries for all UPnP and WSD **AssociationEndpoint** objects when the **DeviceInformationKind** is set to **AssocationEndpoint**.

syntax

```
System.Devices.Aep.ProtocolId="{782232aa-a2f9-4993-971b-aedc551346b0}" OR  
System.Devices.Aep.ProtocolId="{0e261de4-12f0-46e6-91ba-428607ccef64}"
```

This AQS queries for all UPnP **AssociationEndpointService** objects if the **DeviceInformationKind** is set to **AssocationEndpointService**.

syntax

```
System.Devices.AepService.ProtocolId="{0e261de4-12f0-46e6-91ba-428607ccef64}"
```

This AQS queries **AssociationEndpointContainer** objects when the **DeviceInformationKind** is set to **AssociationEndpointContainer**, but only finds them by enumerating the UPnP protocol. Typically, it wouldn't be useful to enumerate containers that only come from one protocol. However, this might be useful by limiting your filter to protocols where you know your device can be discovered.

syntax

```
System.Devices.AepContainer.ProtocolIds:~"{0e261de4-12f0-46e6-91ba-428607ccef64}"
```

Device information properties

Article • 05/06/2023

Each device has associated [DeviceInformation](#) properties that you can use when you need specific information or when you are building a device selector. An Advanced Query Syntax (AQS) filter can be used to specify these properties and limit the enumerated devices with the specified traits. You can also use these properties to indicate what information you want returned for each device. That enables you to specify the device information that is returned to your application.

Important APIs

- [Windows.Devices.Enumeration](#)

For more information about using [DeviceInformation](#) properties in your device selector, see [Build a device selector](#). This topic shows how to request information properties and describes some common properties.

A [DeviceInformation](#) object is composed of an identity ([DeviceInformation.Id](#)), a kind ([DeviceInformation.Kind](#)), and a property bag ([DeviceInformation.Properties](#)). All of the other properties of a [DeviceInformation](#) object are derived from the [Properties](#) property bag. For example, [Name](#) is derived from [System.ItemNameDisplay](#). This means that the property bag always contains the information necessary to determine the other properties.

Requesting properties

A [DeviceInformation](#) object has some basic properties, such as [Id](#) and [Kind](#), but most of the properties are stored in a property bag under [Properties](#). Because of this, the property bag contains the properties used to source the properties out of the property bag. For example, use [System.ItemNameDisplay](#) to source the [Name](#) property. This is a case of a common and well-known property that has a user-friendly name. Windows provides several of these user-friendly names to make querying for properties easier.

When you are requesting properties, you are not limited to the common properties with user-friendly names. You can specify the underlying GUID and property ID (PID) to request any property that is available, even custom properties that are supplied by an individual device or driver. The format for specifying a custom property is "`{GUID} PID`". For example: "`{744e3bed-3684-4e16-9f8a-07953a8bf2ab} 7`".

You can find the list of property GUIDs in the device property key header file of the device driver.

Some properties are common across all [DeviceInformationKind](#) objects, but most are unique to a specific kind. The following sections list some common properties sorted by the individual [DeviceInformationKind](#). For more information about how the different kinds relate to one another, see [DeviceInformationKind](#).

DeviceInterface properties

[DeviceInterface](#) is the default and most common [DeviceInformationKind](#) object used in app scenarios. This is the kind of object that you should use unless the device API indicates a different specific [DeviceInformationKind](#).

 Expand table

Name	Type	Description
System.Devices.ContainerId	GUID	Identity of the DeviceInformationKind.DeviceContainer that contains the Device containing this DeviceInterface . You can pass this value to CreateFromIdAsync along with DeviceInformationKind.DeviceContainer to find the appropriate container.
System.Devices.InterfaceClassGuid	GUID	The interface class GUID this interface represents.
System.Devices.DeviceInstanceId	String	Identity of the parent DeviceInformationKind.Device . You can pass this value to CreateFromIdAsync along with DeviceInformationKind.Device to find the appropriate device.
System.Devices.InterfaceEnabled	Boolean	Indicates if the interface is enabled. DeviceInformation.IsEnabled is derived from this property.
System.Devices.GlyphIcon	String	Icon path for the glyph.
System.Devices.IsDefault	Boolean	Indicates whether this is the default device for the System.Devices.InterfaceClassGuid . This is mainly used for printers. This does not work for audio since there are multiple audio defaults. Use GetDefaultAudioRenderId or GetDefaultAudioCaptureId to get audio defaults.

Name	Type	Description
System.Devices.Icon	String	Icon path.
System.ItemNameDisplay	String	The best display name for the device object.

Device properties

 Expand table

Name	Type	Description
System.Devices.ClassGuid	GUID	Device class used during device installation. For more information, see Device Setup Classes .
System.Devices.CompatibleIds	String[]	The compatible ids of the device. These are used when Windows is determining the best driver to install on the device. For more information, see Compatible ID .
System.Devices.ContainerId	GUID	Identity of the DeviceInformationKind.DeviceContainer that includes this device. You can pass this value to CreateFromIdAsync along with DeviceInformationKind.DeviceContainer to find the appropriate container.
System.Devices.DeviceCapabilities	UInt32	A bitwise-OR of the CM_DEVCAP_X capabilities flags that are defined in CfgMgr32.h . For more information, see DEVPKEY_Device_Capabilities .
System.Devices.DeviceHasProblem	Boolean	The device currently has a problem and is likely not functioning correctly. This could be due to an outdated, missing, or invalid driver.
System.Devices.DeviceInstanceId	String	The identity of the device. This is also the value of DeviceInformation.Id .
System.Devices.DeviceManufacturer	String	The manufacturer of the device.
System.Devices.HardwareIds	String[]	The hardware ids of the device. Windows uses these ids when determining the best driver to install. Device vendors can use this property to identify their device from their app. For more information, see Hardware ID .

Name	Type	Description
System.Devices.Parent	String	The DeviceInformation.Id of the parent device. This is the connection parent, not the DeviceContainer parent.
System.Devices.Present	Boolean	Indicates whether the device is currently present and available.
System.ItemNameDisplay	String	The best display name for this device object. In this case, this is not necessarily the best name for users. A more likely candidate for a user-friendly name could be found by referencing the System.ItemNameDisplay of the associated DeviceContainer or DeviceInterface .

DeviceContainer properties

 Expand table

Name	Type	Description
System.Devices.Category	String[]	A list of descriptions of the categories the device belongs to. This list is provided as singular categories. For example, "Display", "Phone", or "Audio device".
System.Devices.CategoryIds	String[]	Contains a list of categories this device belongs to. For example, Audio.Headphone , Display.Monitor , or Input.Gaming .
System.Devices.CategoryPlural	String[]	A list of descriptions of the categories the device belongs to. This list is provided as plural categories. For example, "Displays", "Phones", or "Audio devices".
System.Devices.CompatibleIds	String[]	The collection of compatible ids for all the child DeviceInformationKind.Device objects.
System.Devices.Connected	Boolean	Indicates whether the device is currently connected to the system or not.
System.Devices.GlyphIcon	String	Icon path for the glyph.
System.Devices.HardwareIds	String[]	The collection of hardware ids for all the child DeviceInformationKind.Device objects.

Name	Type	Description
System.Devices.Icon	String	Icon path.
System.Devices.LocalMachine	Boolean	True if this DeviceContainer represents the system itself, false if the device is external to the system.
System.Devices.Manufacturer	String	The manufacturer of the device.
System.Devices.ModelName	String	Model name of the device container.
System.Devices.Paired	Boolean	Indicates whether any of the child DeviceInformationKind.Device objects are wireless or network devices that are currently paired with the system.
System.ItemNameDisplay	String	The best display name for this device.

DeviceInterfaceClass properties

 Expand table

Name	Type	Description
System.ItemNameDisplay	String	The best display name for this device.

DevicePanel properties

 Expand table

Name	Type	Description
System.Devices.Panel.PanelId	String	The identifier of the DevicePanel object.
System.Devices.Panel.PanelGroup	String	The identifier of the parent PanelGroup .

AssociationEndpoint properties

 Expand table

Name	Type	Description
System.Devices.Aep.AepId	String	Identity of this device. This is also the value of DeviceInformation.Id .
System.Devices.Aep.CanPair	Boolean	Indicates if the device can be paired with the system or not. DeviceInformationPairing.CanPair is derived from this property.
System.Devices.Aep.Category	String[]	The categories that the device is part of. For example, printer or camera.
System.Devices.Aep.ContainerId	GUID	The id of the parent AssociationEndpointContainer object.
System.Devices.Aep.DeviceAddress	String	The address of the device. If the device is a network device, this is the IP address.
System.Devices.Aep.IsConnected	Boolean	Indicates if the device is currently connected to the system.
System.Devices.Aep.IsPaired	Boolean	Indicates if the device is currently paired. DeviceInformationPairing.IsPaired is derived from this property.
System.Devices.Aep.IsPresent	Boolean	Indicates if the device is currently present, meaning the device is live and discovered over the network or wireless protocol. Once a device has been paired with the system, the device is cached. After this, the device will be automatically discovered when querying for AssociationEndpoint objects. Because of this, you cannot rely on just discovering the device with a query to indicate whether or not it is currently usable. That is why this property is important.
System.Devices.Aep.Manufacturer	String	The manufacturer of the device.
System.Devices.Aep.ModelId	GUID	The model id of the device.
System.Devices.Aep.ModelName	String	The model name of the device.
System.Devices.Aep.ProtocolId	GUID	Indicates the protocol used to discover this AssociationEndpoint device.
System.Devices.Aep.SignalStrength	Int32	The signal strength of the device. This property is only applicable for some protocols.
System.ItemNameDisplay	String	The best display name for the device.

AssociationEndpointContainer properties

 Expand table

Name	Type	Description
System.Devices.AepContainer.Categories	String[]	The categories that the device is part of. For example, printer or camera.
System.Devices.AepContainer.Children	String[]	The collection of ids for the AssociationEndpoint objects that are part of this container.
System.Devices.AepContainer.CanPair	Boolean	Indicates if one of the child AssociationEndpoint devices can be paired with the system or not. DeviceInformationPairing.CanPair is derived from this property.
System.Devices.AepContainer.ContainerId	GUID	Identity of this device. This is also the value of DeviceInformation.Id , but in GUID form.
System.Devices.AepContainer.IsPaired	Boolean	Indicates if one of the child AssociationEndpoint devices is currently paired. DeviceInformationPairing.IsPaired is derived from this property.
System.Devices.AepContainer.IsPresent	Boolean	Indicates if one of the child AssociationEndpoint devices is currently present, meaning the device is live and discovered over the network or wireless protocol. Once a device has been paired with the system, the device is cached. After this, the device will be automatically discovered when querying for AssociationEndpoint objects. Because of this, you cannot rely on just discovering the device with a query to indicate whether or not it is currently usable. That is why this property is important.

Name	Type	Description
System.Devices.AepContainer.Manufacturer	String	The manufacturer of the device.
System.Devices.AepContainer.ModelIds	String[]	A list of model ids for the device. Each model is a GUID in string form.
System.Devices.AepContainer.ModelName	String	The model name of the device.
System.Devices.AepContainer.ProtocolIds	GUID[]	A list of protocol ids that have contributed to building this AssociationEndpointContainer object. Keep in mind that an AssociationEndpointContainer device is created by collecting all AssociationEndpoint devices discovered over different protocols for the same physical device.
System.Devices.AepContainer.SupportedUriSchemes	String[]	List of casting URI schemes supported by this device.
System.Devices.AepContainer.SupportsAudio	Boolean	Indicates if this device supports audio casting.
System.Devices.AepContainer.SupportsImages	Boolean	Indicates if this device supports image casting.
System.Devices.AepContainer.SupportsVideo	Boolean	Indicates if this device supports video casting.
System.ItemNameDisplay	String	The best display name for the device.

AssociationEndpointService properties

 Expand table

Name	Type	Description
System.Devices.AepService.AepId	String	The identifier of the parent AssociationEndpoint object.
System.Devices.AepService.ContainerId	GUID	The identifier of the parent AssociationEndpointContainer

Name	Type	Description
		object.
System.Devices.AepService.ParentAepIsPaired	Boolean	Indicates whether the parent AssociationEndpoint object is paired with the system.
System.Devices.AepService.ProtocolId	GUID	Identity of the protocol used to discover this device.
System.Devices.AepService.ServiceClassId	GUID	Identity of the service represented by this device.
System.Devices.AepService.ServiceId	String	Identity of this service. This is also the value of DeviceInformation.Id .
System.ItemNameDisplay	String	The best display name for the service.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

AEP service class IDs

Article • 05/06/2023

Association Endpoint (AEP) services provide a programming contract for services that a device supports over a given protocol. Several of these services have established identifiers that should be used when referencing them. These contracts are identified with the **System.Devices.AepService.ServiceClassId** property. This topic lists several well-known AEP service class IDs. The AEP service class ID is also applicable to protocols with custom class IDs.

An app developer should use Advanced Query Syntax (AQS) filters based on the class IDs to limit their queries to the AEP services they plan to use. This will both limit the query results to the relevant services and will significantly increase the performance, battery life, and quality of service for the device. For example, an application can use these service class IDs to use a device as a Miracast sync or DLNA digital media renderer (DMR). For more information about how devices and services interact with each other, see [DeviceInformationKind](#).

Important APIs

- [Windows.Devices.Enumeration](#)

Bluetooth and Bluetooth LE services

Bluetooth services fall under one of two protocols, either the Bluetooth protocol or the Bluetooth LE protocol. The identifiers for these protocols are:

- Bluetooth protocol ID: {e0cbf06c-cd8b-4647-bb8a-263b43f0f974}
- Bluetooth LE protocol ID: {bb7bb05e-5972-42b5-94fc-76eaa7084d49}

The Bluetooth protocol supports several services, all following the same basic format. The first group of digits in the GUID vary based upon the service, but all Bluetooth GUIDs end with **-0000-1000-8000-00805F9B34FB**. For example, the RFCOMM service has the precursor of 0x0003, so the full ID would be **00000003-0000-1000-8000-00805F9B34FB**. The following table lists some common Bluetooth services.

Service name	GUID
RFCOMM	00000003-0000-1000-8000-00805F9B34FB
GATT - Alert notification service	00001811-0000-1000-8000-00805F9B34FB

Service name	GUID
GATT - Automation IO	00001815-0000-1000-8000-00805F9B34FB
GATT - Battery service	0000180F-0000-1000-8000-00805F9B34FB
GATT - Blood pressure	00001810-0000-1000-8000-00805F9B34FB
GATT - Body composition	181B0000-0000-1000-8000-00805F9B34FB
GATT - Bond management	0000181E-0000-1000-8000-00805F9B34FB
GATT - Continuous glucose monitoring	0000181F-0000-1000-8000-00805F9B34FB
GATT - Current time service	00001805-0000-1000-8000-00805F9B34FB
GATT - Cycling power	00001818-0000-1000-8000-00805F9B34FB
GATT - Cycling speed and cadence	00001816-0000-1000-8000-00805F9B34FB
GATT - Device information	0000180A-0000-1000-8000-00805F9B34FB
GATT - Environmental sensing	0000181A-0000-1000-8000-00805F9B34FB
GATT - Generic access	00001800-0000-1000-8000-00805F9B34FB
GATT - Generic attribute	00001801-0000-1000-8000-00805F9B34FB
GATT - Glucose	00001808-0000-1000-8000-00805F9B34FB
GATT - Health thermometer	00001809-0000-1000-8000-00805F9B34FB
GATT - Heart rate	0000180D-0000-1000-8000-00805F9B34FB
GATT - Human interface device	00001812-0000-1000-8000-00805F9B34FB
GATT - Immediate alert	00001802-0000-1000-8000-00805F9B34FB
GATT - Indoor positioning	00001821-0000-1000-8000-00805F9B34FB
GATT - Internet protocol support	00001820-0000-1000-8000-00805F9B34FB
GATT - Link loss	00001803-0000-1000-8000-00805F9B34FB
GATT - Location and navigation	00001819-0000-1000-8000-00805F9B34FB
GATT - Next DST change service	00001807-0000-1000-8000-00805F9B34FB
GATT - Phone alert status service	0000180E-0000-1000-8000-00805F9B34FB
GATT - Pulse oximeter	00001822-0000-1000-8000-00805F9B34FB
GATT - Reference time update service	00001806-0000-1000-8000-00805F9B34FB

Service name	GUID
GATT - Running speed and cadence	00001814-0000-1000-8000-00805F9B34FB
GATT - Scan parameters	00001813-0000-1000-8000-00805F9B34FB
GATT - Tx power	00001804-0000-1000-8000-00805F9B34FB
GATT - User data	0000181C-0000-1000-8000-00805F9B34FB
GATT - Weight scale	0000181D-0000-1000-8000-00805F9B34FB

For a more complete listing of available Bluetooth services, see the [GATT Services specification](#) [↗]. You can also use the [GattServiceUuids](#) API to get some common GATT services.

Custom Bluetooth LE services

Custom Bluetooth LE services use the following protocol identifier: {bb7bb05e-5972-42b5-94fc-76eaa7084d49}

Custom profiles are defined with their own defined GUIDs. This custom GUID should be used for `System.Devices.AepService.ServiceClassId`.

UPnP services

UPnP services use the following protocol identifier: {0e261de4-12f0-46e6-91ba-428607ccef64}

In general, all UPnP services have their name hashed into a GUID using the algorithm defined in RFC 4122. The following table lists some common UPnP services defined in Windows.

Service name	GUID
Connection manager	ba36014c-b51f-51cc-bf71-1ad779ced3c6
AV transport	deeacb78-707a-52df-b1c6-6f945e7e25bf
Rendering control	cc7fe721-a3c7-5a14-8c49-4419dc895513
Layer 3 forwarding	97d477fa-f403-577b-a714-b29a9007797f
WAN common interface configuration	e4c1c624-c3c4-5104-b72e-ac425d9d157c
WAP IP connection	e4ac1c23-b5ac-5c27-8814-6bd837d8832c

Service name	GUID
WFA WLAN configuration	23d5f7db-747f-5099-8f21-3ddfd0c3c688
Printer enhanced	fb9074da-3d9f-5384-922e-9978ae51ef0c
Printer basic	5d2a7252-d45c-5158-87a4-05212da327e1
Media receiver registrar	0b4a2add-d725-5198-b2ba-852b8bf8d183
Content directory	89e701dd-0597-5279-a31c-235991d0db1c
DIAL	085dfa4a-3948-53c7-a0d7-16d8ec26b29b

WSD services

WSD services use the following protocol identifier: {782232aa-a2f9-4993-971b-aedc551346b0}

In general, all WSD services have their name hashed into a GUID using the algorithm defined in RFC 4122. The following table lists some common WSD services defined in Windows.

Service name	GUID
Printer	65dca7bd-2611-583e-9a12-ad90f47749cf
Scanner	56ec8b9e-0237-5cae-aa3f-d322dd2e6c1e

AQS sample

This AQS will filter for all UPnP **AssociationEndpointService** objects that support DIAL. In this case, **DeviceInformationKind** is set to **AssocationEndpointService**.

syntax

```
System.Devices.AepService.ProtocolId="{0e261de4-12f0-46e6-91ba-428607ccef64}" AND
System.Devices.AepService.ServiceClassId="{085DFA4A-3948-53C7-A0D7-16D8EC26B29B}"
```

Pair devices

Article • 05/06/2023

This topic describes how to pair devices using the [Windows.Devices.Enumeration](#) namespace.

Important APIs

- [Windows.Devices.Enumeration](#)

Automatic pairing

Sometimes you want to use a device in your application, but do not care whether or not the device is paired. You simply want to be able to use the functionality associated with a device. For example, if your app wants to simply capture an image from a webcam, you are not necessarily interested in the device itself, just the image capture. If there are device APIs available for the device you are interested in, this scenario would fall under automatic pairing.

In this case, you simply use the APIs associated with the device, making the calls as necessary and trusting the system to handle any pairing that might be necessary. Some devices do not need to be paired in order for you to use their functionality. If the device does need to be paired, then the device APIs will handle the pairing action behind the scenes so you do not need to integrate that functionality into your app. Your app will have no knowledge about whether or not a given device is paired or needs to be, but you will still be able to access the device and use its functionality.

Basic pairing

Basic pairing is when your application uses the [Windows.Devices.Enumeration](#) APIs in order to attempt to pair the device. In this scenario, you are letting Windows attempt the pairing process and handle it. If any user interaction is necessary, it will be handled by Windows. You would use basic pairing if you need to pair with a device and there is not a relevant device API that will attempt automatic pairing. You just want to be able to use the device and need to pair with it first.

In order to attempt basic pairing, you first need to obtain the [DeviceInformation](#) object for the device you are interested in. Once you receive that object, you will interact with the [DeviceInformation.Pairing](#) property, which is a [DeviceInformationPairing](#) object. To attempt to pair, simply call [DeviceInformationPairing.PairAsync](#). You will need to await

the result in order to give your app time to attempt to complete the pairing action. The result of the pairing action will be returned, and as long as no errors are returned, the device will be paired.

If you are using basic pairing, you also have access to additional information about the pairing status of the device. For example you know the pairing status (**IsPaired**) and whether the device can pair (**CanPair**). Both of these are properties of the **DeviceInformationPairing** object. If you are using automatic pairing, you might not have access to this information unless you obtain the relevant **DeviceInformation** objects.

Custom pairing

Custom pairing enables your app to participate in the pairing process. This allows your app to specify the **DevicePairingKinds** that are supported for the pairing process. You will also be responsible for creating your own user interface to interact with the user as needed. Use custom pairing when you want your app to have a little more influence over how the pairing process proceeds or to display your own pairing user interface.

In order to implement custom pairing, you will need to obtain the **DeviceInformation** object for the device you are interested in, just like with basic pairing. However, the specific property you are interested in is **DeviceInformation.Pairing.Custom**. This will give you a **DeviceInformationCustomPairing** object. All of the **DeviceInformationCustomPairing.PairAsync** methods require you to include a **DevicePairingKinds** parameter. This indicates the actions that the user will need to take in order to attempt to pair the device. See the **DevicePairingKinds** reference page for more information about the different kinds and what actions the user will need to take. Just like with basic pairing, you will need to **await** the result in order to give your app time to attempt to complete the pairing action. The result of the pairing action will be returned, and as long as no errors are returned, the device will be paired.

To support custom pairing, you will need to create a handler for the **PairingRequested** event. This handler needs to make sure to account for all the different **DevicePairingKinds** that might be used in a custom pairing scenario. The appropriate action to take will depend on the **DevicePairingKinds** provided as part of the event arguments.

It is important to be aware that custom pairing is always a system-level operation. Because of this, when you are operating on Desktop or Windows Phone, a system dialog will always be shown to the user when pairing is going to happen. This is because both of those platforms possess a user experience that requires user consent. Since that dialog is automatically generated, you will not need to create your own dialog when you are

opting for a [DevicePairingKinds](#) of **ConfirmOnly** when operating on these platforms. For the other **DevicePairingKinds**, you will need to perform some special handling depending on the specific **DevicePairingKinds** value. See the sample for examples of how to handle custom pairing for different **DevicePairingKinds** values.

Starting with Windows 10, version 1903, a new **DevicePairingKinds** is supported, **ProvidePasswordCredential**. This value means that the app must request a user name and password from the user in order to authenticate with the paired device. To handle this case, call the [AcceptWithPasswordCredential](#) method of the event args of the **PairingRequested** event handler to accept the pairing. Pass in a [PasswordCredential](#) object that encapsulates the user name and password as a parameter. Note that the username and password for the remote device are distinct from and often not the same as the credentials for the locally signed-in user.

Unpairing

Unpairing a device is only relevant in the basic or custom pairing scenarios described above. If you are using automatic pairing, your app remains oblivious to the pairing status of the device and there is no need to unpair it. If you do choose to unpair a device, the process is identical whether you implement basic or custom pairing. This is because there is no need to provide additional information or interact in the unpairing process.

The first step to unpairing a device is obtaining the [DeviceInformation](#) object for the device that you want to unpair. Then you need to retrieve the [DeviceInformation.Pairing](#) property and call [DeviceInformationPairing.UnpairAsync](#). Just like with pairing, you will want to **await** the result. The result of the unpairing action will be returned, and as long as no errors are returned, the device will be unpaired.

Sample

For a working sample, see the [Device enumeration and pairing sample](#) [↗](#) on GitHub.

Pairing a set with the DeviceInformationPairing.Custom property

Article • 03/27/2024

ⓘ Note

Some information relates to pre-released product, which may be substantially modified before it's commercially released. Microsoft makes no warranties, express or implied, with respect to the information provided here.

Use the [Windows.Devices.Enumeration](#) APIs to pair devices as a set.

Windows supports pairing a collection of devices as a set. The platform supports two types of set.

- **Automatically-discovered set.** This happens when a protocol (such as Bluetooth LE) automatically discovers other endpoints that belong to the set while pairing a primary endpoint. For example, while a user is pairing the right earbud over Bluetooth LE, the protocol stack might discover the left earbud; so both can be paired together as a set.
- **Explicit set.** These are useful when a device is discovered over more than one protocol. For example, Internet Printing Protocol (IPP) printers are typically discovered over three protocols: IPP, WSD, and eSCL. When multiple endpoints for the same device are discovered, then they can be explicitly paired together as a set.

Code example for an automatically-discovered set (Bluetooth-style)

This code example implements automatically-discovered (Bluetooth-style) set pairing using a custom pairing object. As with a typical implementation of custom pairing, a *pairing-requested handler* is required to handle the pairing ceremony. In this case, the code example implements only the *confirm only* pairing ceremony. The new and interesting part is adding a *pairing-set-member-requested* handler.

The set-member-handler enables the platform to attempt pairing the passed-in device as a set. Without that handler, the protocol stack won't attempt to enumerate the

members of a pairing set. The Bluetooth protocol discovers set members as part of finalizing the pairing, so the set handler is called sometime after its ceremony handler returns. In this flow, the set handler generally can expect to be called once after handling the ceremony with a fixed list of set members; meaning that the protocol discovers all of the endpoints that it can during pairing, and it won't discover more later.

This code example synchronously pairs all of the discovered set members with the same **BtSetPairing** routine used to pair the primary device/endpoint. Pairing them in parallel is also supported, and might be more efficient for your scenario. But for simplicity, that's not shown in the code example. Since we're also pairing the set members with a set handler, they can potentially recursively produce more set members to pair. But typically, the set handler for the discovered set members will probably only see *discovery completed* with the set member vector being empty.

ⓘ Note

This code example is without the context of a larger scenario or app; but an app would probably need to track what devices it's pairing, and the pairing results. That's in order for the app to get a sense of whether the whole set-pairing operation was successful or not.

ⓘ Note

These APIs are generally asynchronous. Pairing operations have their own worker threads, and handlers are called on different threads. Your code doesn't have to block as often as this code example does.

Code example in C++/WinRT

C++/WinRT

```
void PairingTests::BtSetPairing(DeviceInformation device)
{
    DevicePairingKinds ceremonies = DevicePairingKinds::ConfirmOnly;
    auto customPairing = device.Pairing().Custom();
    event_revoker ceremonyEventToken = customPairing.PairingRequested(
        { this, &PairingTests::PairingRequestedHandler });
    event_revoker setEventToken = customPairing.PairingSetMembersRequested(
        { this, &PairingTests::PairingSetMembersRequestedHandler });

    DevicePairingResult result = customPairing.PairAsync(ceremonies).get();

    if (DevicePairingResultStatus::Paired == result.Status()) // Pairing
```

```

worked.
    else // Handle pairing failure.
}

void PairingTests::PairingRequestedHandler(DeviceInformationCustomPairing
const&,
    DevicePairingRequestedEventArgs const& args)
{
    switch (args.PairingKind())
    {
    case DevicePairingKinds::ConfirmOnly:
        args.Accept();
        break;
    default:
        // This code example doesn't implement other ceremonies.
        // The handler wouldn't be called for ceremonies that the app didn't
register.
    }
}

void
PairingTests::PairingSetMembersRequestedHandler(DeviceInformationCustomPairi
ng
    const&, DevicePairingSetMembersRequestedEventArgs const& args)
{
    switch (args.Status())
    {
    case
DevicePairingAddPairingSetMemberStatus::SetDiscoveryCompletedByProtocol:
        // This is the expected result if we started set pairing
        // a Bluetooth device. Note: there still might be no set members.
        break;
    case
DevicePairingAddPairingSetMemberStatus::SetDiscoveryPartiallyCompletedByProt
ocol:
        // The protocol enumerated some but not all set members.
        break;
    case
DevicePairingAddPairingSetMemberStatus::SetDiscoveryNotAttemptedByProtocol:
        // Not expected for Bluetooth.
        // This constant implies that the protocol likely doesn't support
set-pairing.
    default:
        // The other constants aren't expected in an auto-discovered
Bluetooth set scenario.
        // Error handling can go here.
    }

    for (auto setMember : args.PairingSetMembers())
    {
        BtSetPairing(setMember);
    }
}

```

Code example for an explicit set (IPP-style)

This code example implements explicit set pairing using a custom pairing object. As with a typical implementation of custom pairing, a *pairing-requested handler* is required to handle the pairing ceremony. In this case, the code example implements only the *confirm only* pairing ceremony. As with the Bluetooth code example, the new and interesting part is adding a *pairing-set-member-requested* handler. The set member handler enables the platform to attempt pairing devices as a set.

As compared to the Bluetooth-style set-pairing scenario, this code example explicitly adds devices to the set. And the set handler implies something a bit different for the protocols related to pairing IPP printers. It implies that clients are handling device discovery over the various protocols, and the PnP state and print queues created as a result of pairing all of the set members should be synchronized.

To keep the implementation of the code example simple, it assumes that a vector of set member endpoints were discovered beforehand, and passed in as a parameter along with the primary device. For example, in a typical IPP scenario, endpoints are discovered in arbitrary order. So the primary device could have been discovered over WSD for instance; and then the vector would contain devices representing endpoints discovered over IPP, and eSCL. But any combination is possible and valid. It adds set members to the primary device's custom pairing object on the app's main thread, and then calls [PairAsync](#).

ⓘ Note

In practice, set members might be added to a custom pairing object at any time on any thread. Operations over a protocol can take a long time, or might even be blocked until they time out, so overlapping them is useful. Consider taking advantage of the parallelism of the API to add and pair devices concurrently. This is possible even while you're still enumerating devices over the wire. The advantages of pairing them as a set still generally apply.

With this implementation, the primary set member will be paired concurrently with the set members. The set members are paired one at a time synchronously in the handler. But again, they can be paired in parallel for better efficiency.

Device node objects in PnP are often created as a result of pairing. In the case of IPP, device nodes are always created for each endpoint after pairing. This set-pairing API implicitly synchronizes device node creation between endpoints in the set. In this code example's flow, all device nodes will be synchronized because all set members are

added before pairing starts. For more detail on how this API synchronizes device nodes in PnP, see the [General commentary](#) section in this topic.

ⓘ Note

This code example is without the context of a larger scenario or app; but an app would probably need to track what devices it's pairing, and the pairing results. That's in order for the app to get a sense of whether the whole set-pairing operation was successful or not.

Code example in C++/WinRT

C++/WinRT

```
void PairingTests::IppSetPairing(DeviceInformation device,
    std::vector<DeviceInformation> const& setMemberDevices)
{
    DevicePairingKinds ceremonies = DevicePairingKinds::ConfirmOnly;
    auto customPairing = device.Pairing().Custom();
    event_revoker ceremonyEventToken = customPairing.PairingRequested({
this,
        &PairingTests::PairingRequestedHandler });
    event_revoker setEventToken = customPairing.PairingSetMembersRequested({
this,
        &PairingTests::PairingSetMembersRequestedHandler });

    if (setMemberDevices)
    {
        for (auto setDevice : setMemberDevices)
        {
            customPairing.AddPairingSetMember(setDevice);
        }
    }

    DevicePairingResult result = customPairing.PairAsync(ceremonies).get();

    if (DevicePairingResultStatus::Paired == result.Status()) // Pairing
worked.
    else // Handle pairing failure.
    }

void PairingTests::PairingRequestedHandler(DeviceInformationCustomPairing
const&,
    DevicePairingRequestedEventArgs const& args)
{
    switch (args.PairingKind())
    {
    case DevicePairingKinds::ConfirmOnly:
        args.Accept();
    }
```

```

        break;
    }
}

void
PairingTests::PairingSetMembersRequestedHandler(DeviceInformationCustomPairing const&,
        DevicePairingSetMembersRequestedEventArgs args)
{
    switch (args.Status())
    {
    case DevicePairingAddPairingSetMemberStatus::AddedToSet:
        // This is the expected result of adding a set member(s)
        // by calling the AddPairingSetMember method.
        break;
    case DevicePairingAddPairingSetMemberStatus::CouldNotBeAddedToSet:
        // Means we failed to add set member(s).
        break;
    case
DevicePairingAddPairingSetMemberStatus::SetDiscoveryNotAttemptedByProtocol:
    default:
        // The other constants aren't expected in an explicit set scenario.
        // Error handling can go here.
    }

    for (auto setMember : args.PairingSetMembers())
    {
        IppSetPairing(setMember, nullptr);
    }
}

```

General commentary

Automatically-discovered (Bluetooth-style) set pairing

This API is necessary in order to accomplish Bluetooth-style set pairing where set members are discovered by the protocol after the primary endpoint is paired. A simple example might be a set of wireless earbuds. As the first earbud's pairing is finalized, the device informs the PC that there's a second device to pair as part of the set. The custom pairing API is extended to enable apps to handle set operations through the new *add set member status* handler.

Explicit (IPP-style) set pairing

Similarly, you can also use this API to pair any group of association endpoint (AEP) devices as a set. With a custom pairing object, your app can add other endpoints to the pairing set at any time on any thread. That's by design because device discovery and

pairing over a network can take a long time for each device, so we don't want to serialize these operations when we can avoid it.

It's especially useful to pair in sets when discovering devices over many protocols where the protocol and device stacks aren't easily coordinated. For example, a modern network printer will likely be discovered by Windows with three different network protocols, each of which produces association endpoints. In that case, pairing all three endpoints as a set is very useful for a couple of reasons: it avoids wasteful rediscovery over the wire, and it creates a single streamlined print queue.

Even if a network printer isn't paired as a set, the print spooler still attempts to create a single print queue per user regardless of whether it can be paired over several protocols. If a printer is initially paired over one protocol, then the operating system (OS) will attempt to rediscover it over the other supported protocols, and associate over all of them in order to avoid duplicate print queues. Typically, the OS can do that quickly and successfully and produce one streamlined print queue.

However, if the app has already discovered all the endpoints for the printer, then this rediscovery step is wasteful. And worse, it can add long delays before the printer is ready to use. Also, if protocols are too out-of-sync or delayed, then the spooler might have to create extra print queues for the same printer, which is confusing to end users.

Pairing all endpoints at once as a set avoids potentially slow rediscovery. It ensures that the PnP state is synchronized, and produces optimal streamlined print queues.

Device node synchronization

When the devices are paired as a set with this API, the resulting PnP device state is conveniently synchronized. The API doesn't constrain *when* an app can add set member; but there are limits on when the platform can synchronize device nodes. Device node synchronization blocks until all endpoints in the set have their pairing finalized. After that, all device nodes for all endpoints are created at once. You can add more set members to the set after that point, but no subsequent device node creation is blocked, but is instead created right away.

- Device node creation *is* synchronized when:
 - Set members are added before pairing starts.
 - Set members are added while at least one of the set members is not finalized.
- Device node creation is *not* synchronized:
 - Any time after all added set members have been finalized.

As a practical matter, the API doesn't allow an app to control when the ceremony finalization completes in order to influence this behavior on how device nodes are

synchronized. The closest approximation would be when the app chooses to complete the ceremony. A pairing can't finalize until the app's ceremony handler returns; so that's the app's last chance to influence when all of the set members are finalized.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Dynamic lighting

Article • 12/12/2024

This topic describes how your Windows apps can provide dynamic lighting effects across connected devices that implement the open [Human Interface Devices \(HID\) Lighting and Illumination standard](#). In particular, the LampArray specification for devices that have one or more lamps (lights, LEDs, bulbs, and so on).

Important APIs

- [Windows.Devices.Lights](#)

ⓘ Note

The [Lamp](#) class does not apply to HID [LampArray](#) functionality and is not relevant to this topic.

Overview

Dynamic lighting lets both Windows app developers and end users control and synchronize lighting effects across compatible peripherals and other connected devices. These features can be used to deliver cohesive experiences that entertain users, make them more productive, and even make their experiences across Windows more accessible.

Supported app types and platforms

- Win10 version 1809 (October 2018) and later.
 - Applies to foreground UWP and Win32 apps.
- Windows 11 Build 23466 (Preview) and later.
 - Applies to foreground and background (ambient) UWP and Win32 apps.
- Xbox GDK March 2023 Update 1 and later.
 - See the [Lighting API](#) in the [Game Development Kit \(GDK\)](#).

Supported devices and device types

- Keyboard or keypad
- Mouse
- Game controller (gamepad, flightstick, steering wheel, and so on).
- Peripheral (general devices such as speakers, mouse pads, microphones, webcams, and so on).

- Scene (room/stage/area devices such as light bulbs, spotlights, strobe lights, billboards, camera flashes, and so on).
- Notification (user attention devices such as alarms, voice assistants, and so on).
- Chassis (internal PC components such as RAM, motherboard, fan, and so on).
- Wearable (accessories such as headsets, watches, fitness trackers, shoes, and so on).
- Furniture (such as chairs, desks, bookcases, and so on).
- Art (such as a painting or sculpture).
- Headset (accessories designed specifically for the head, such as headphones or microphones).

A Windows app can control HID LampArray devices when the app is in the foreground (starting with Windows 10) and when it's in the background (also known as *ambient lighting*, starting with Windows 11).

Device prioritization

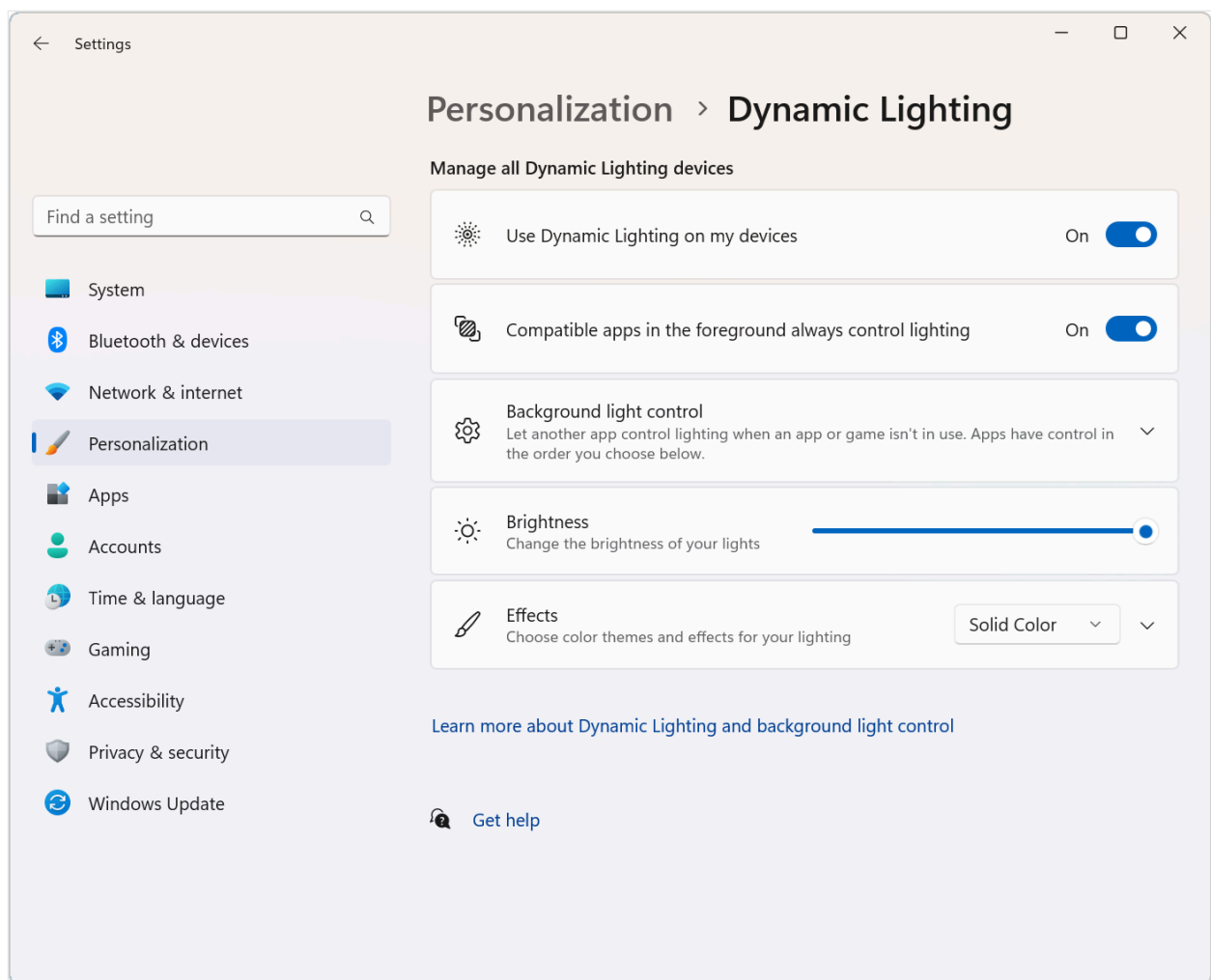
Windows prioritizes dynamic lighting based on app state. By default, a foreground app is always assigned control of a LampArray device unless the user has specified otherwise in Settings. In cases where two or more ambient background apps are trying to control a LampArray device, the system will assign control to the app prioritized in Settings.

Background (ambient) lighting

The "ambient" APIs in [Windows.Devices.Lights](#) enable background applications to control LampArray devices while the user is interacting with an unrelated app in the foreground (such as music apps that drive synchronized lighting effects).

Apps can receive [LampArray.AvailabilityChanged](#) events, depending on user settings. In conjunction with the [DeviceWatcher](#) class, apps can track and manage all connected/disconnected LampArray devices and see which the user expects the app to control. One example usage is a UI that renders an icon for each connected device, unavailable ones greyed out, and links to the **Dynamic Lighting** settings page where the user can make changes to foreground/background app preferences.

User settings



Users can customize their LampArray device experience (both foreground and background) through the **Settings -> Personalization -> Dynamic lighting** screen.

1. When connected, compatible devices will show up in the device cards along the top of the page where users can change individual device settings.
2. Global Dynamic Lighting settings are located below the device cards (changes to these settings affect all connected devices).
 - a. The *Use Dynamic Lighting on my devices* toggle lets users turn Dynamic Lighting on or off. When Dynamic Lighting is off, devices should function with their default non-Dynamic Lighting behavior. Dynamic lighting includes a built-in set of basic effects.
 - b. The *Compatible apps in the foreground always control lighting* allows users to turn the default Dynamic Lighting app behavior on or off. When this feature is toggled off, a background app can control its associated devices even when a foreground app that wants control is active.
 - c. The *Background light control* section lets users prioritize installed apps that have registered themselves as ambient background controllers. Dragging an app to the top of the list will prioritize it and ensure that it can control devices over other apps in the list. Ambient background settings are tied to a device and the